

Coding Section

16 October 2025 08:23

Pattern Based Coding Questions

<https://www.edureka.co/blog/30-pattern-programs-in-java/>

a. Pyramid Pattern

```
12      int n = 5;
13      for(int i=1; i<= n; i++ ){
14
15          for(int j= n -i; j>= 1 ; j--){
16              System.out.print(" ");
17          }
18
19          for(int j=0; j < i ; j++){
20              System.out.print("* ");
21          }
22          System.out.println();
23      }
24  }
```

Output:

```
Output Clear
java -cp /tmp/yPmUxYESRs/HelloWorld
*
* *
* * *
* * * *
* * * * *
```

a. Diamond Pattern

Hint: Pyramid's upper part code will remain the same only lower part logic code needs to add

```
26      // Lower block code
27      for(int p=n-1; p >= 1; p-- ){
28
29          for(int j=n-p; j >=1 ; j--){
30              System.out.print(" ");
31          }
32
33          for(int j=0; j <p ; j++){
34              System.out.print("* ");
35          }
36
37          System.out.println();
38      }
```

Output:

```
Output Clear
java -cp /tmp/Gtlbfiz7I2/HelloWorld
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * *
* * *
* *
*
```

a. Left Triangle Pattern

Hint: If we remove space code block from Pyramid pattern code then left triangle pattern will print

Output :

```
Output Clear

java -cp /tmp/9xpqghYlsh/HelloWorld
*
* *
* * *
* * * *
* * * * *
```

a. Right Triangle Pattern

Hint: If we remove little space from "*" " from pyramid source code then it will print right triangle pattern

```
8- class HelloWorld {
9-     public static void main(String[] args) {
10         // System.out.println("Try programiz.pro");
11
12         int n = 5;
13         for(int i=1; i<= n; i++){
14
15             for(int j= n -i; j>= 1 ; j--){
16                 System.out.print(" ");
17             }
18
19             for(int j=0; j < i ; j++){
20                 System.out.print("*");
21             }
22             System.out.println();
23         }
24     }
25 }
```

Output:

```
Output Clear

java -cp /tmp/D7JbMFMdYA/HelloWorld
*
**
***
****
*****
```

Reg Ex Based coding Questions

<https://www.hackerrank.com/domains/regex>

<https://runestone.academy/ns/books/published/py4e-int/regex/hp-practice-adv.html>

<https://www.interviewbit.com/coding-interview-questions/>

Stream API

Just watch Telusko video on Stream API.

<https://javaconceptsoftheday.com/java-8-stream-intermediate-and-terminal-operations/>

<https://stackify.com/streams-guide-java-8/>

<https://www.jrebel.com/blog/java-streams-cheat-sheet>

<https://www.linkedin.com/pulse/how-safely-use-map-operation-java-8-streams-nikhil-gargatte/>

<https://www.digitalocean.com/community/tutorials/java-stream-collect-method-examples>

<https://www.educative.io/answers/what-is-streamsorted-method-in-java>

1. Converting a Collection to Stream

- Collections like `List`, `Set`, or `Map` can be converted to a stream using the `stream()` method.

java

Copy code

```
List<String> list = Arrays.asList("apple", "banana", "cherry");  
Stream<String> stream = list.stream();
```

2. Converting Arrays to Stream

- Arrays can be converted to a stream using `Arrays.stream()` or `Stream.of()` methods.

java

Copy code

```
String[] array = {"apple", "banana", "cherry"};  
Stream<String> stream = Arrays.stream(array);  
  
int[] intArray = {1, 2, 3};  
IntStream intStream = Arrays.stream(intArray); // for primitive types
```



3. Converting Primitives to Stream

- For primitive types, Java provides specialized streams: `IntStream`, `LongStream`, and `DoubleStream`.

java

Copy code

```
IntStream intStream = IntStream.of(1, 2, 3, 4);  
LongStream longStream = LongStream.of(1L, 2L, 3L);  
DoubleStream doubleStream = DoubleStream.of(1.0, 2.0, 3.0);
```

4. Converting Strings to Stream

- Strings can be converted into a stream of characters or into other formats using `chars()` and `split()` methods.

java

Copy code

```
String str = "Hello";  
IntStream charStream = str.chars(); // Stream of int representing Unicode of characters  
  
Stream<String> wordStream = Arrays.stream(str.split(" ")); // Split string into a stream
```

5. Converting a Stream of Different Data Types

- If you have mixed types in a list, you can filter them using `instanceof` and convert them to their specific streams.

```
java                                                                    Copy code

List<Object> mixedList = Arrays.asList(1, "apple", 2, "banana");

Stream<Integer> intStream = mixedList.stream()
    .filter(e -> e instanceof Integer)
    .map(e -> (Integer) e);

Stream<String> stringStream = mixedList.stream()
    .filter(e -> e instanceof String)
    .map(e -> (String) e);
```

Coding Questions should be know before every interview

Important Import packages in Java before starting

```
import java.util.List;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.Map;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.stream.Collectors;
import java.util.Collections;
import java.util.Optional;
```

To find min and max in Collections object

<https://www.geeksforgeeks.org/finding-minimum-and-maximum-element-of-a-collection-in-java/>

1. Iterator --> Remove element using Iterator | Done

<https://www.geeksforgeeks.org/iterators-in-java/>

1. Stream API --> sort element based on age in map having name and age as key value pair | Done

Why to use entrySet() ?

<https://www.naukri.com/code360/library/hashmap-entryset-method-in-java>

Updated code for sorting HashMap

```
Map<String,Integer> sortedMap = map.entrySet()
    .stream()
    .sorted(Map.Entry.comparingByValue())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1,e2) -> e1, LinkedHashMap::new));
```

Output:

Output

Clear

```
java -cp /tmp/WViEQUjhjF/HelloWorld
Sorted Map : {12=3, 13=1, 14=2}
```

1. Duplicate letter in a string | Done

Mississippi : return letter and it's count

<https://www.javatpoint.com/java-program-to-count-occurrences-of-each-character>

<https://stackoverflow.com/questions/10462819/get-keys-from-hashmap-in-java>

<https://www.geeksforgeeks.org/hashmap-values-method-in-java/>

```
// To get all the values
System.out.println(map.values());

// To get all the keys

for(char ch: map.keySet())
{
    System.out.println(ch);
}
```

1. To check palindrome --> can't reverse and store | Done

<https://www.geeksforgeeks.org/java-program-to-check-whether-a-string-is-a-palindrome/>

Easiest and simple method using StringBuilder method

<https://medium.com/@brandon93.w/stringbuilder-and-stringbuffer-in-java-examples-and-interview-exercises-fd735f15c415#:~:text=This%20is%20an%20unsynchronized%2C%20mutable,create%20multiple%20intermediate%20string%20objects.>

1. Code to give second highest element of the array without sorting | Done

<https://www.geeksforgeeks.org/find-second-largest-element-array/>

```
static int getSecondLargest(int[] arr) {
    int n = arr.length;

    int largest = -1, secondLargest = -1;

    // Finding the largest element
    for (int i = 0; i < n; i++) {
        if (arr[i] > largest)
            largest = arr[i];
    }

    // Finding the second largest element
    for (int i = 0; i < n; i++) {

        // Update second largest if the current element is greater
        // than second largest and not equal to the largest
        if (arr[i] > secondLargest && arr[i] != largest) {
            secondLargest = arr[i];
        }
    }
    return secondLargest;
}
```

Using Stream API Method:

```
// How to do from Stream API
Optional<Integer> streamLargest = list.stream()
    .sorted( (a,b) -> b-a)
    .skip(1)
    .findFirst();

System.out.println(streamLargest);
```

1. Reverse string (use stringbuilder or stringbuffer)--> Completed

<https://www.geeksforgeeks.org/stringbuffer-class-in-java/>

1. Print unique elements from the list --> Completed | Done

- Using Stream API
- Get max and min using Stream API
- Sort it out using Stream API

```
class Main {
    public static void main(String[] args) {
        System.out.println("Distinct number from List");

        // Initialise list
        List<Integer> list1= List.of(34,67,12,45,32);

        List<Integer> list = new ArrayList<>(List.of(34,67,12,45,32));
        list.add(12);
        list.add(12);
        list.add(23);
        list.add(56);
        list.add(45);
        list.add(56);

        List<Integer> list2= list.stream()
            .distinct()
            .collect(Collectors.toList());

        System.out.println("Uniques/Distinct Number: " + list2);

        // How to max and min using stream API

        int max = list2.stream()
            .max(Integer::compare)
            .get();

        int min = list2.stream()
            .min(Integer::compare)
            .get();

        System.out.println("Maximum Number: " + max);
        System.out.println("Minimum Number: " + min);
    }
}
```

Output:

```
Distinct number from List
Uniques/Distinct Number: [34, 67, 12, 45, 32, 23, 56]
Maximum Number: 67
Minimum Number: 12
```

1. Second largest element from the list using java 8 stream API | Done

```
12- public class HelloWorld {
13-     public static void main(String[] args) {
14-
15-         ArrayList<Integer> list = new ArrayList<>();
16-
17-         list.add(12);
18-         list.add(142);
19-         list.add(22);
20-         list.add(42);
21-         list.add(52);
22-         list.add(102);
23-
24-         // System.out.println(list);
25-
26-         Optional<Integer> secondLargest = list.stream()
27-             .sorted((a,b) -> b-a)
28-             .skip(1)
29-             .findFirst();
30-
31-         System.out.println("Second Highest element: "+ secondLargest);
32-
33-     }
34- }
```

Output:

Output

Clear

```
java -cp /tmp/3RGDTK4PDJ/HelloWorld
Second Highest element: Optional[102]
```

1. Using hashmap print the number of times element is repeating as value and the element as key(e.g. List of (12,12,12,14) | Done

Key -12, value 3

Key -14, value 1

```
16      List<Integer> list = new ArrayList<>();
17      list.add(12);
18      list.add(14);
19      list.add(14);
20      list.add(13);
21      list.add(12);
22      list.add(12);
23      |
24      HashMap<Integer,Integer> map = new HashMap<>();
25
26 -      for( int no : list){
27 -          if(map.containsKey(no)){
28 -              int count = map.get(no);
29 -              map.put(no, ++count);
30 -          }else{
31 -              map.put(no,1);
32 -          }
33      }
34      System.out.println("Updated Map : " + map);
35  }
36 }
```

Output:

Output

Clear

```
java -cp /tmp/xyA8JcQqFE/HelloWorld
Updated Map :{12=3, 13=1, 14=2}
```

1. Swap two numbers without using temporary variable | Done

<https://www.w3schools.in/java/examples/swapping-two-numbers-without-using-a-temporary-variable>

```
public class JavaSwapExample {

    public static void main(String[] args) {

        int x = 10;
        int y = 20;

        System.out.println("Before Swapping");
        System.out.println("Value of x is : " + x);
        System.out.println("Value of y is : " + y);

        //add both the numbers and assign it to first
        x = x + y;
        y = x - y;
        x = x - y;

        System.out.println("Before Swapping");
        System.out.println("Value of x is : " + x);
        System.out.println("Value of y is : " + y);

    }
}
```

1. Distinct character count in string (Use hashmap) | Done

```

11- public class HelloWorld {
12-     public static void main(String[] args) {
13-
14-         String str = "Mississippi";
15-
16-         HashMap<Character,Integer> map = new HashMap<>();
17-         // Character and it's count
18-         for(int i=0; i< str.length()-1; i++){
19-
20-             if(map.containsKey(str.charAt(i))){
21-                 int count = map.get(str.charAt(i));
22-                 map.put(str.charAt(i), ++count);
23-             }else{
24-                 map.put(str.charAt(i), 1);
25-             }
26-         }
27-
28-         // List it down
29-         System.out.println(map);
30-     }
31- }

```

Output:

Output

Clear

```

java -cp /tmp/HafxXDdatE/HelloWorld
{p=2, s=4, i=3, M=1}

```

1. **Prove string is immutable --> Completed | Done**

Example 1: <https://stackoverflow.com/questions/27902471/program-to-prove-string-is-immutable>

String objects created using string literals are stored in the String Constant Pool and any two objects in the pool can't have the same content.

Before modifications, they are pointing to the same object. Once we tried to change the content of the object using 's1', a new object is created in the pool and its reference is assigned to s1. If the strings are mutable, both s1 and s2 should point to the same object even after modification.

```

public class MyClass {
    public static void main(String[] args) {

        String s1 = "java";
        String s2 = "java";

        System.out.println("No modification is made");

        referenceCheck(s1,s2);

        s1 = s1.concat("code");

        System.out.println("After modification");

        referenceCheck(s1,s2);

    }

    public static void referenceCheck(Object x, Object y)
    {
        if(x==y)
        {
            System.out.println("Both objects points to same reference");
        }

        else
        {
            System.out.println("Both pointing to different object");
        }
    }
}

```

Example 2:

```

4- class HelloWorld {
5-     public static void main(String[] args) {
6-         // System.out.println("Try programiz.pro");
7-
8-         String a = "abc";
9-         String b = a;
10-        a = a.concat("d");
11-        System.out.println(a);
12-        System.out.println(b);
13-
14-        StringBuffer c = new StringBuffer("abc");
15-        StringBuffer d = c;
16-        c.append("d");
17-        System.out.println(c);
18-        System.out.println(d);
19-
20-    }
21- }

```

Output:

Output

Clear

```

java -cp /tmp/JiPu4QwMVe/HelloWorld
abcd
abc
abcd
abcd

```

Explanation:

a. String

- **Explanation:**

- `String a = "abc";` : A `String` object with the value `"abc"` is created, and `a` references this object.
- `String b = a;` : Now, `b` references the same object as `a`. So both `a` and `b` point to `"abc"`.
- `a = a.concat("d");` : A new `String` object `"abcd"` is created because `String` objects in Java are **immutable** (cannot be changed after they are created). The variable `a` is now updated to reference this new object `"abcd"`, but `b` still references the old `"abc"`.
- When you print `a`, it prints `"abcd"`, and when you print `b`, it prints `"abc"` because `b` still points to the original `Str` ↓.

a. StringBuffer

- **Explanation:**

- `StringBuffer c = new StringBuffer("abc");` : A `StringBuffer` object is created with the value `"abc"`, and `c` references this object.
- `StringBuffer d = c;` : Now, `d` references the same `StringBuffer` object as `c`.
- `c.append("d");` : This modifies the original `StringBuffer` object by appending `"d"` to the end of `"abc"`, resulting in `"abcd"`. Since `StringBuffer` is **mutable** (it can be modified after creation), no new object is created. Both `c` and `d` still reference the same modified object.
- When you print `c` and `d`, both print `"abcd"` because they reference the same object, which has been modified in place.

a. **Show Diamond problem with multiple inheritance | Done**

<https://www.prepbytes.com/blog/java/diamond-problem-in-java/>

Inheritance in Java

The diamond problem in java occurs because of inheritance so before directly moving to understand the diamond problem in java let's first have a brief discussion about inheritance in java.

Inheritance is basically a concept with the help of which the child class can inherit the features, attributes, and methods of the parent class. This allows the developers to create a hierarchy of classes where the subclasses are inherited from the parent classes.

Multiple inheritance is nothing but when a child inherits the properties of more than one parent class.

```

3- class Sample1{
4-     public void sum(int x,int y){
5-         System.out.println("Sum= "+ x+y);
6-     }
7- }
8-
9- class Sample2{
10-     public void sum(int x,int y){
11-         System.out.println("Sum= "+ x+y);
12-     }
13- }
14-
15- public class HelloWorld extends Sample1, Sample2{
16-     public static void main(String[] args) {
17-         // System.out.println("Try programiz.pro");
18-
19-         Dummy1 obj = new Dummy1();
20-         obj.sum();
21-     }
22- }
23- }

```

Output:

Output

Clear

```

ERROR!
/tmp/HQBALLnI5k/HelloWorld.java:16: error: '{' expected
public class HelloWorld extends Sample1, Sample2{
                                   ^
1 error

```

- Reverse string without in built method and sort
Hint: Iterate over string and traverse from last
- Employee class is there in which 3 attributes are there (id, name, salary) and return second highest salary | Done

<https://www.jrebel.com/blog/java-streams-cheat-sheet>

<https://www.linkedin.com/pulse/how-safely-use-map-operation-java-8-streams-nikhil-gargatte/>

<https://www.digitalocean.com/community/tutorials/java-stream-collect-method-examples>

Important Imports:

```

import java.util.List;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.HashMap;
import java.util.stream.Collectors;
import java.util.Comparator;

```

```

58- public class HelloWorld {
59-     public static void main(String[] args) {
60-         |
61-         // Initialise list
62-         List<Employee> list = new ArrayList<>();
63-
64-         list.add( new Employee( 12 , "Sandy", 23000 ) );
65-         list.add( new Employee( 22 , "Maxwell", 123000.50 ) );
66-         list.add( new Employee( 32 , "David", 2443000 ) );
67-         list.add( new Employee( 42 , "Clark", 263000.50 ) );
68-         list.add( new Employee( 52 , "Java", 723000 ) );
69-
70-         // Use stream api to sort list
71-         List<Employee> sortedList = list.stream()
72-             .sorted(Comparator.comparing(e -> e.getSalary()))
73-             .collect(Collectors.toList());
74-
75-         // display
76-         sortedList.forEach(System.out::println);
77-
78-     }
79- }

```

Output:

Output

[Clear](#)

```
java -cp /tmp/tIevQYdCUL/HelloWorld
Employee{id='12', name=Sandy, salary=23000.0}
Employee{id='22', name=Maxwell, salary=123000.5}
Employee{id='42', name=Clark, salary=263000.5}
Employee{id='52', name=Java, salary=723000.0}
Employee{id='32', name=David, salary=2443000.0}
```

1. Sort hashmap by value --> Completed | Done

Why to use entrySet() ?

<https://www.naukri.com/code360/library/hashmap-entryset-method-in-java>

Updated code for sorting HashMap by Value

```
Map<String,Integer> sortedMap = map.entrySet()
    .stream()
    .sorted(Map.Entry.comparingByValue())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1,e2) -> e1, LinkedHashMap::new));
```

Output:

Output

[Clear](#)

```
java -cp /tmp/WViEQUjhjF/HelloWorld
Sorted Map : {12=3, 13=1, 14=2}
```

1. input string --> convert into map --> 1 as key :one as value

String input = "1:one,2:two,3:three";

```
import java.util.Arrays;
import java.util.Map;
import java.util.stream.Collectors;

public class StringToMap {
    public static void main(String[] args) {
        String input = "1:one,2:two,3:three";

        // Convert the string to a Map
        Map<Integer, String> map = Arrays.stream(input.split(","))
            .map(entry -> entry.split(":"))
            .collect(Collectors.toMap(
                entry -> Integer.parseInt(entry[0]), // Key: Convert string to integer
                entry -> entry[1] // Value: Use the string as-is
            ));

        // Print the map
        map.forEach((key, value) ->
            System.out.println("Key: " + key + ", Value: " + value));
    }
}
```

2. two integer array --> merge array --> without duplicate

```
// array1 : 1,2,3,4
// array2 : 4,3,7,2,1
```

```

15     int[] arr1 = { 1,2,3,4,5,6};
16     int[] arr2 = { 4,5,6,7,8};
17
18     // Arrays can be converted into stream
19
20     // Approach 1: Convert array to list --> use stream() method to convert
    into stream
21
22     IntStream stm1 = Arrays.stream(arr1);
23     IntStream stm2 = Arrays.stream(arr2);
24
25     IntStream stm3 = IntStream.concat(stm1 , stm2 );
26
27     stm3.forEach(str -> System.out.print(str + " "));
28
29 }
30 }

```

Output:

Output

Clear

```

java -cp /tmp/guD1B514yH/HelloWorld
1 2 3 4 5 6 4 5 6 7 8

```

Reference:

<https://stackoverflow.com/questions/61297074/using-java-streams-to-merge-a-pair-of-int-arrays>

<https://howtodoinjava.com/java8/stream-concat-example/#:~:text=Java%20%20Stream-,The%20Java%20%20Stream.,also%20be%20a%20parallel%20stream.>

▲

3

▼

Here is one way, using `Stream::flatMapToInt`.

```

int[] a = { 10 , 20 , 30 , 40 };
int[] b = { 50 , 60 , 70 , 80 };
int[] merged = Stream.of(a,b).flatMapToInt(IntStream::of)
    .toArray();

```

3. `int[] arr = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};`

/* Expected Output

[0, 1, 2]

[3, 4, 5]

[6, 7, 8]

[9]

*/

```

11
12     int splitSize = 3;
13
14     int[] arr = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
15
16     List<Object> list = new ArrayList<>();
17
18     int pointer = 0 ;
19
20     while ( pointer < arr.length ) {
21         HashSet<Integer> set = new HashSet<>();
22         for ( int i=pointer; i< splitSize; i++){
23             if ( pointer < arr.length) {
24                 set.add(arr[i]);
25                 pointer = pointer + 1;
26             }
27         }
28         list.add(set);
29         splitSize = splitSize + 3;
30     }
31
32     for (Object object : list) {
33         System.out.println(object);
34     }
35 }

```

Output:

```

[0, 1, 2]
[3, 4, 5]
[6, 7, 8]
[9]

```

1. Input:

```

name = [ "Akshay", "Neha", "Snehar", "Vipin" ]
roll_no = [ 4, 1, 3, 2 ]
marks = [ 40, 50, 60, 70 ]

```

Output:

The zipped result is : (('Akshay', 4, 40), ('Snehar', 3, 60), ('Vipin', 2, 70), ('Neha', 1, 50))

```

18     List<Object> combined = new ArrayList<>();
19
20     String[] name = { "Akshay", "Neha", "Snehar", "Vipin" };
21     int[] roll_no = { 4, 1, 3, 2 };
22     int[] marks = { 40, 50, 60, 70 };
23
24     // Approach: 1. arraylist initialise 2. customize string 3. iterate
25
26     for(int i =0 ; i< name.length; i++){
27         String temp="";
28         temp += "(" + name[i] + "," + roll_no[i] + "," + marks[i] + ")";
29         combined.add(temp);
30     }
31     System.out.println(combined);

```

Output:

```

Output
Clear

java -cp /tmp/ddKwIPUTNc/HelloWorld
[(Akshay,4,40), (Neha,1,50), (Snehar,3,60), (Vipin,2,70)]

```

1. Some questions based on Reg Ex

Input

Regular expressions go one step further: They allow you to specify a pattern of text to search for. You may not know a business's exact phone number, but if you live in

India, you know it will be two/three digits of state code , followed by a hyphen or space, and then eight more digits.

This is how you, a human would know a phone number when he see it:
022-25277413

```
8- public static void main(String[] args) {
9-     // System.out.println("Hello World");
10
11     // Input: 022-25277413
12     // Pattern: two/three digits of state code, followed by a hyphen or
        space, and then eight more digits.
13
14     String input = "022-25277413";
15     // 1. split on '-'/space 2. check two/three digit --> state code 3.
        check if eight digit
16
17     String[] newStr = input.split("-");
18
19     for(Object object : newStr){
20
21         if ( object.toString().length() == 2 || object.toString().length()
            ==3 ){
22             System.out.println("State : "+ object);
23         }else{
24             System.out.println("Phone number length : "+ object);
25         }
26     }
27 }
28 }
```

Output:

```
Output Clear
java -cp /tmp/to20wePshu/HelloWorld
State : 022
Phone number length : 25277413

=== Code Execution Successful ===
```

Using Reg Ex Approach

https://www.w3schools.com/java/java_regex.asp

```
5- import java.util.*;
6- import java.util.regex.Matcher;
7- import java.util.regex.Pattern;
8
9- class HelloWorld {
10-     public static void main(String[] args) {
11
12         String input = "022-25277413";
13         // 1. split on '-'/space 2. check two/three digit --> state code 3.
            check if eight digit
14
15         Pattern pattern = Pattern.compile("[0-9]{3}-[0-9]{8}");
16         Matcher matcher = pattern.matcher(input);
17
18         boolean matchFound = matcher.find();
19
20         if(matchFound){
21             System.out.println("Match Found");
22         }
23     }
24 }
```

Output:

```
Output Clear
java -cp /tmp/tHSUFsEGB7/HelloWorld
Match Found
```

Create a regex that finds dates in the format MM/DD/YY or MM/DD/YYYY and returns just the year part.

-- [0-9]{2}/[0-9]{2}/[1-9]{4}

Create a regex that finds phone numbers like 333-232-3403 or (333) 232 3403.

-- [1-9]{3}-[1-9]{3}-[1-9]{3}

Replace 'YOUR_REGEX' below with a regex that matches any price in the form of \$3.45 or \$23.32 or \$400.

```
Main.java
7 import java.util.regex.Pattern;
8
9- class HelloWorld {
10-     public static void main(String[] args) {
11
12         String input = "$3.45";
13         // $3.45 or $23.32 or $400.
14         // 1. split on '-'/space 2. check two/three digit --> state code 3.
15         //    check if eight digit
16
17         Pattern pattern = Pattern.compile("^\\$[0-9]+(\\. [0-9]{2})?*$");
18         Matcher matcher = pattern.matcher(input);
19
20         boolean matchFound = matcher.find();
21
22         if(matchFound){
23             System.out.println("Match Found");
24         }else{
25             System.out.println("Match Not Found");
26         }
27     }
28 }
```

Output:

```
Output
Clear

java -cp /tmp/wBu0j1Nuk9/HelloWorld
Match Found
```

Explanation:

1. `^\\$`: The `^` asserts the start of the string, and `\\$` matches the literal dollar sign `$`. The backslash is used to escape the dollar sign, as it is a special character in regex.
2. `[0-9]+`: Matches one or more digits (the whole number part).
3. `(\\. [0-9]{2})?`:
 - `\\.` matches the literal dot `.` (decimal point).
 - `[0-9]{2}` matches exactly two digits (the fractional part).
 - The entire group `(\\. [0-9]{2})?` is optional, allowing for whole numbers without a fractional part.
4. `$`: Asserts the end of the string.

Construct a regex that captures words that start with a vowel (a, e, i, o, u), can have any number of characters from a-z after the vowel (including zero), and end with a consonant (any letters that are not a, e, i, o, u). For example, it should match "unicorn", "it", and "element".

```
Main.java
3
4 // import java.util.HashSet;
5- import java.util.*;
6 import java.util.regex.Matcher;
7 import java.util.regex.Pattern;
8
9- class HelloWorld {
10-     public static void main(String[] args) {
11
12         String input = "element";
13
14         Pattern pattern = Pattern.compile("^[aeiou]([a-z]*)(^[^aeiou]$)");
15         Matcher matcher = pattern.matcher(input);
16
17         boolean matchFound = matcher.find();
18
19         if(matchFound){
20             System.out.println("Match Found");
21         }else{
22             System.out.println("Match Not Found");
23         }
24     }
25 }
```

Explanation:

Explanation:

1. `^[aeiou]`:

- `^`` asserts the start of the string.
- `[aeiou]` matches any single vowel (a, e, i, o, u) at the start of the string.

2. `[a-z]*`:

- Matches zero or more characters from a-z (any lowercase letter).

3. `^[^aeiou]$`:

- `[^aeiou]` matches any character that is not a vowel, i.e., a consonant.
- `$`` asserts the end of the string.

2. To count no of digit in Integer| Done

```
Main.java
13 // array2 : 43721
14
15 public static void main(String[] args) {
16
17     int input = 12345;
18     int count = 0;
19
20     // Approach 1: Convert into string and find length
21     String op = Integer.toString(input);
22
23     System.out.println("Count using String method : " + op.length());
24
25     // Approach 2: Modulo method
26 while (input > 0) {
27     count++;
28     input = input/10;
29 }
30
31 System.out.println("Count using Modulo method: " + count);
32
33 }
34 }
```

Output:

```
Output
Clear

java -cp /tmp/DxU252pfU1/HelloWorld
Count using String method : 5
Count using Modulo method: 5
```

3. list of fruits filter out starts with 'M'
4. positive no at the end and negative number at the start
5. Program to parenthesis balancing

<https://www.geeksforgeeks.org/check-for-balanced-parentheses-in-an-expression/>

```
String str = "[()]" ;
```

```
Step 1: Declare stack which hold previous bracket
Stack<Character> stk = new Stack<>();
```

```
Step 2: check if character is an opening bracket -> Push to stack
```

```
for( int i=0; i< str.length(); i++)
```

```
{
    if(str.charAt(i) == '(' || str.charAt(i) == '{' || str.charAt(i) == '[')
    {
        stk.push(str.charAt(i));
    }
```

```
Step 3: if it is non empty and closing bracket then check it is matching opening bracket -> Pop from stack
```

```
else
{
    if( !stk.empty() && (stk.peek() == ')' || str.charAt(i) == '}')
        stk.pop()
    else
        return false;
}
```

```
}
```

6. find all the triplet from the given list of no that sum up to the target
7. Common longest Substring in List of type string

```

class Main {
    public static void main(String[] args) {
        System.out.println("Try programiz.pro");

        List<String> list = new ArrayList<>();
        list.add("Flower");
        list.add("Flask");
        list.add("Floor");

        System.out.println(list);
        String longSubstr = list.get(0);

        for(int i=1; i< list.size(); i++)
        {
            while ( list.get(i).indexOf(longSubstr)!=0 )
            {
                longSubstr = longSubstr.substring(0, longSubstr
                    .length() -1);
            }
        }

        System.out.println(longSubstr);
    }
}

```

Output:

```

Try programiz.pro
[Flower, Flask, Floor]
Fl

```

8. Recursion problem

<https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/examples-Java-recursion-recursive-methods>

9. how you can find the employee with the highest salary using Java

```

public class HighestSalaryFinder {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Alice", 60000),
            new Employee("Bob", 75000),
            new Employee("Charlie", 90000),
            new Employee("Diana", 90000) // Same as Charlie
        );

        Optional<Employee> highestPaid = employees.stream()
            .max(Comparator.comparingDouble(Employee::getSalary));

        if (highestPaid.isPresent()) {
            System.out.println("Highest paid employee: " + highestPaid.get().getName() +
                " with salary: " + highestPaid.get().getSalary());
        }
    }
}

```

```
List<String> topEarnings = employees.stream()
    .filter(e -> e.getSalary() == maxSalary)
    .map(Employee::getName)
    .collect(Collectors.toList());
```

10. Rotate the array by k unit

For example, if nums = [1,2,3,4,5,6,7] and k = 3, the output should be [5,6,7,1,2,3,4].

```
public class RotateArray {
    public static void rotate(int[] nums, int k) {
        int n = nums.length;
        k = k % n; // Handle k > n

        reverse(nums, 0, n - 1); // Step 1: Reverse whole array
        reverse(nums, 0, k - 1); // Step 2: Reverse first k elements
        reverse(nums, k, n - 1); // Step 3: Reverse the rest
    }

    private static void reverse(int[] nums, int start, int end) {
        while (start < end) {
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start++;
            end--;
        }
    }
}
```

11. Count the number of palindromic substrings in a string

```
public class Main {
    public static void main(String[] args) {
        String str = "abcccdc";
        int n = str.length();

        int count = 0;

        // Generate all substrings
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                String substr = str.substring(i, j + 1);

                // Check if substring is a palindrome
                String reversed = new StringBuilder(substr).reverse().toString();
                if (substr.equals(reversed)) {
                    count++;
                }
            }
        }

        System.out.println("Number of palindromic substrings: " + count);
    }
}
```

12. Java 8 Questions --> Important for EY

Implement a Stack Using Arrays

Find the Missing Number

FizzBuzz Problem

Reference: <https://javaconceptsoftheday.com/java-8-interview-sample-coding-questions/>

Given a list of strings, find out those strings which start with a number?

How do you find common elements between two arrays?

<https://www.geeksforgeeks.org/find-common-elements-in-two-arrays-in-java/>

```
// Find the common elements
list1.retainAll(list2);
```

Java 8 | How do you merge two unsorted arrays into single sorted array without duplicates?

```
import java.util.List;
import java.util.ArrayList;
import java.util.Map;
import java.util.HashMap;
import java.util.Arrays;
import java.util.stream.IntStream;

public class Main {
    public static void main(String[] args) {
        System.out.println("Try programiz.pro");
        int[] a = new int[] {4, 2, 5, 1};
        int[] b = new int[] {8, 1, 9, 5};

        int[] c = IntStream.concat(Arrays.stream(a), Arrays.stream(b))
            .sorted().distinct().toArray();

        System.out.println(Arrays.toString(c));
    }
}
```

Steps:

- Concat two arrays
- Sort both
- Distinct for uniqueness
- Convert to Array

Find the age of a person in years if the birthday has given?

<https://www.geeksforgeeks.org/how-to-calculate-age-from-a-birthdate-using-java-date-time/>

```
import java.time.LocalDate;
import java.time.Period;
```

```
// To get age if given birthdate
LocalDate birthday = LocalDate.of(1997,10,15);
LocalDate currentDate = LocalDate.now();

Period period = Period.between(birthday, currentDate);

System.out.println("Age: " + period.getYears());
```

Output

Clear

Try programiz.pro
Age: 27

Java 8 | Given an integer array, find sum and average of all elements?

```
// To get average of list of element  
double average = list.stream()  
    .mapToDouble(Integer::doubleValue)  
    .average()  
    .orElse(0.0);
```

Output

Clear

Try programiz.pro
[1, 2, 3, 5, 6, 7, 8, 9]
5.125

Given a list of strings, sort them according to increasing order of their length?

Java 8 | Find sum of all digits of a number in Java 8?

```
List<Integer> list = Arrays.asList(1,2,3,5,6,7,8,9);  
  
long sum = list.stream()  
    .reduce((a,b) -> a+b)  
    .get();
```

Try programiz.pro
[1, 2, 3, 5, 6, 7, 8, 9]
41

Java 8 | Program to check if two strings are anagrams or not?

```
import java.util.stream.Collectors;  
import java.util.stream.Stream;  
  
public class Java8Code  
{  
    public static void main(String[] args)  
    {  
        String s1 = "RaceCar";  
        String s2 = "CarRace";  
  
        s1 = Stream.of(s1.split("")).map(String::toUpperCase).sorted().collect(Collectors.joining());  
        s2 = Stream.of(s2.split("")).map(String::toUpperCase).sorted().collect(Collectors.joining());  
  
        if (s1.equals(s2))  
        {  
            System.out.println("Two strings are anagrams");  
        }  
        else  
        {  
            System.out.println("Two strings are not anagrams");  
        }  
    }  
}
```

Steps:

- Use steam.of() method and pass list of type String using split method
- Map() with UpperCase
- Sorted
- Collect() joining

Java 8 | Find the longest string in a list of strings

Very Important Exercise: <https://www.benchresources.net/java-8-find-longest-string-in-an-arrays-or-list-or-stream/>

✓ Equivalent Stream API Version:

java

Copy

Edit

```
String maxLen = list.stream()
    .max(Comparator.comparingInt(String::length))
    .orElse(null);
```

Java 8 | Find the length of longest string in a list of strings

✓ 2. Finding the Length of the Longest String (Integer Result)

java

Copy

Edit

```
int lengthOfLongestString = names
    .stream()
    .map(String::length)
    .max(Integer::compare)
    .get();
```

Java 8 | sum of square of odd number using Stream API

```
int[] arr = {1,2,3,4...10};
```

```
int result = IntStream.of(arr)
    .filter(n -> n % 2 != 0)    // Keep only odd numbers
    .map(n -> n * n)           // Square them
    .sum();                    // Sum it up
```

```
// To get sum of square of even number
double squareOfEven = list.stream()
    .filter( e -> e%2==0)
    .map( n-> n*n)
    .reduce( (a,b) -> a+b)
    .get();
```

Output

Clear

```
Try programiz.pro
[1, 2, 3, 5, 6, 7, 8, 9]
104.0
```

Java 8 | IMP | Find the Most Frequent Character in a String (excluding spaces)

```
Optional<Character> result = input.chars()
    .mapToObj(c -> (char) c)
    .filter(c -> c != ' ')
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey);
```

Java 8 | IMP | Group Employees by Department and List Names Alphabetically

```
java Copy Edit

Map<String, List<String>> result = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
        Collectors.mapping(Employee::getName,
            Collectors.collectingAndThen(Collectors.toList(),
                list -> list.stream().sorted().collect(Collectors.toList()))));
```

- Use collect() method
- group by the stream based on department (groupingby() methods of stream) -> it return stream of map Map<K, List<>>

```
{
    "HR" -> [Employee("Alice"), Employee("Charlie"), Employee("Eva")],
    "IT" -> [Employee("Bob"), Employee("David")]
}
```

Operation	Output	
1. Count by Department	{'HR': 3, 'IT': 2}	employees.stream() .collect(Collec
2. Sorted Names by Department	{'HR': ['Alice', 'Charlie', 'Eva'], 'IT': ['Bob', 'David']}	employees.stream() .collect(Collec
3. Senior-most Employee by Department	{'HR': 'Charlie', 'IT': 'David'}	employees.stream() .collect(Collec
4. Average Salary by Department	{'HR': 64333.33, 'IT': 77000}	employees.stream() .collect(Collec
5. Departments with >2 Employees	{'HR': ['Alice', 'Charlie', 'Eva']}	grouped.entrySet().stream() .filter

- Some more operation can be perform based on above output

```
java Copy Edit

{
    "HR": ["Alice", "Charlie", "Eva"],
    "IT": ["Bob", "David"]
}
```

- Different approach to solve the problem

```
// 2. Group Employees by Department and List Names Alphabetically
Map<String, List<String>> depToNames =
list.stream()
    .collect(Collectors.groupingBy
        (e -> e.getDepartment(),
            Collectors.mapping(e -> e.getName(), Collectors.toList
                (())));

// sorted list of Names in each department
depToNames.replaceAll((dept, name) -> name.stream().sorted().collect
    (Collectors.toList()));
```

Java 8 | IMP | FlatMap a List of Sentences into Unique Sorted Words

```
// 3. FlatMap a List of Sentences into Unique Sorted Words
String[] str = {"Java is fun", "Streams are powerful", "Fun with Java"};

List<String> list = new ArrayList<>(Arrays.asList(str));

System.out.println(list);

List<String> result =
list.stream()
    .flatMap(e -> Arrays.stream(e.split(" ")))
    .map(String::toLowerCase)
    .distinct()
    .sorted()
    .collect(Collectors.toList());

// Output: ["are", "fun", "is", "java", "powerful", "streams", "with"]
```

Why Use Arrays.stream(...)?

Because `.split(" ")` gives us a `String[]`, **but we need a Stream** for `flatMap`.

So:

- `sentence.split(" ")` → returns an array (`String[]`)
- `Arrays.stream(...)` → converts that array into a `Stream<String>`

Now that each sentence gives a **Stream of words**, we can combine them.

Java 8 | IMP | Count Word Frequency from List of Strings

```
List<String> words = Arrays.asList("apple", "banana", "apple", "orange", "banana", "apple");

Map<String, Long> wordCount = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Java 8 | IMP | Nested Grouping: Group by Department, Then by Age

java

Copy

Edit

```
List<Employee> employees = Arrays.asList(
    new Employee("Alice", "HR", 30),
    new Employee("Bob", "IT", 35),
    new Employee("Charlie", "HR", 30),
    new Employee("David", "IT", 35),
    new Employee("Eva", "HR", 25)
);
```

Output Structure:

The resulting map looks like:

java

Copy

Edit

```
{
  "HR" = {
    25 = [Eva],
    30 = [Alice, Charlie]
  },
  "IT" = {
    35 = [Bob, David]
  }
}
```

Layer	Grouping Key	Result Type
Outer Map	String (department)	→ Map<String, Map<Integer, List>>
Inner Map	Integer (age)	→ Map<Integer, List>

From <<https://chatgpt.com/c/68243bc5-3598-8005-ad90-b3410e0d8135>>

java

Copy

Edit

```
Map<String, Map<Integer, List<Employee>>> grouped = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment, // First group by department
        Collectors.groupingBy(Employee::getAge) // Then group by age within department
    ));
```

Why does input.chars() return an IntStream, not Stream<Character>?

Because in Java, **char** is a **primitive type**, and Java Streams are divided into:

- **Object streams:** Stream<T>
- **Primitive streams:** IntStream, LongStream, DoubleStream

Since char is **not a wrapper class** (like Character), and there's **no CharStream**, String.chars() returns an IntStream of Unicode code points.

So why do we convert int to char?

1 input.chars()

- Converts the **String** to an **IntStream** of Unicode values of each character.

java

Copy

Edit

```
"abc" → 97, 98, 99
```

2 .mapToObj(c -> (char) c)

- Converts each **int** back to a **char**.

java

Copy

Edit

```
97 → 'a', 98 → 'b', etc.
```

Data structure & Algorithm Questions

<https://workat.tech/problem-solving/practice/topics>

<https://sathishsuresh.medium.com/cracking-data-structures-coding-interview-abca041dbd59>

a. LinkedList Operations

- To initialise the LinkedList

```
class ListNode
{
    int val;
    ListNode next;

    ListNode(int val)
    {
        this.val = val;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("Try programiz.pro");

        ListNode head = new ListNode(1);
        head.next = new ListNode(2);
        head.next.next = new ListNode(3);
        head.next.next.next = new ListNode(4);

        Main.printList(head);
        System.out.println("-----Printing Reverse LinkedList -----");
        Main.printReverse(head);
    }
}
```

- To print the element of LinkedList

```
public static void printList(ListNode node)
{
    while(node!=null)
    {
        System.out.print(node.val + " -> ");
        node = node.next;
    }
    System.out.println("null");
}
```

- Reverse logic

```

public ListNode reverseList(ListNode head)
{
    ListNode prev = null, curr = head;

    while(curr != null)
    {
        ListNode nextTemp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}

```

- To print the element of reverse LinkedList

```

public static void printReverse(ListNode node)
{
    if(node == null) return ;
    printReverse(node.next);
    System.out.print(node.val + "-> ");
}

```

- To find loop in LinkedList

<https://www.interviewbit.com/blog/detect-loop-in-linked-list/>

Algorithm:

- Initialise two pointers, **fast** and **slow** to the **head** of the linked list.
- Traverse through the linked list until the **fast** pointer doesn't reach the end of the linked list.
- If the **fast** pointer reaches the end, it means that the linked list doesn't contain any cycle. Hence, return **False**.
- Else, move the **slow** pointer by one node i.e. **slow = slow → next** and **fast** pointer by two nodes i.e. **fast = fast → next → next**.
- At any point, if the **fast** and the **slow** pointers point to the same node, return **True** as a loop has been detected.

Java Code for Two Pointer Approach

```
// JAVA Code
public boolean hasCycle(ListNode head) {
    if (head == null) {
        return false;
    }
    ListNode slow = head;
    ListNode fast = head.next;
    do {
        if (fast == null || fast.next == null) {
            return false;
        }
        slow = slow.next;
        fast = fast.next.next;
    } while (slow != fast);
    return true;
}
```

b. Tree Operations

- Insert elements in the Tree

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) {
        this.val = val;
    }
}

public class BinarySearchTree {
    TreeNode root;

    public TreeNode insert(TreeNode node, int val) {
        if (node == null) return new TreeNode(val);
        if (val < node.val)
            node.left = insert(node.left, val);
        else
            node.right = insert(node.right, val);
        return node;
    }
}
```

- Traversal operations

Reference:

<https://www.geeksforgeeks.org/tree-traversals-inorder-preorder-and-postorder/>

```

public void inorder(TreeNode node) {
    if (node != null) {
        inorder(node.left);
        System.out.print(node.val + " ");
        inorder(node.right);
    }
}

public static void main(String[] args) {
    BinarySearchTree bst = new BinarySearchTree();
    int[] values = {5, 3, 7, 2, 4};
    for (int val : values)
        bst.root = bst.insert(bst.root, val);

    System.out.print("Inorder: ");
    bst.inorder(bst.root); // Output: 2 3 4 5 7
}

```

c. Graph Operation

- Adjacency List (Using HashMap)

```

import java.util.*;

public class Graph {
    private Map<Integer, List<Integer>> adjList = new HashMap<>();

    // Add an edge to the graph
    public void addEdge(int u, int v, boolean directed) {
        adjList.putIfAbsent(u, new ArrayList<>());
        adjList.get(u).add(v);

        if (!directed) {
            adjList.putIfAbsent(v, new ArrayList<>());
            adjList.get(v).add(u);
        }
    }

    // Print the graph
    public void printGraph() {
        for (int node : adjList.keySet()) {
            System.out.println(node + " -> " + adjList.get(node));
        }
    }
}

```

```

public static void main(String[] args) {
    Graph graph = new Graph();
    graph.addEdge(0, 1, false);
    graph.addEdge(0, 2, false);
    graph.addEdge(1, 2, false);
    graph.addEdge(2, 3, false);

    graph.printGraph();
}

```

- Adjacency Matrix (2D Array)

```
public class GraphMatrix {  
    int[][] matrix;  
    int nodes;  
  
    public GraphMatrix(int n) {  
        nodes = n;  
        matrix = new int[n][n];  
    }  
  
    public void addEdge(int u, int v) {  
        matrix[u][v] = 1;  
        matrix[v][u] = 1; // for undirected graph  
    }  
  
    public void printGraph() {  
        for (int i = 0; i < nodes; i++) {  
            System.out.println(Arrays.toString(matrix[i]));  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    GraphMatrix g = new GraphMatrix(4);  
    g.addEdge(0, 1);  
    g.addEdge(0, 2);  
    g.addEdge(1, 2);  
    g.addEdge(2, 3);  
  
    g.printGraph();  
}
```

- Weighted Graph using Adjacency List

```
class Edge {  
    int to, weight;  
    Edge(int to, int weight) {  
        this.to = to;  
        this.weight = weight;  
    }  
}  
  
public class WeightedGraph {  
    private Map<Integer, List<Edge>> graph = new HashMap<>();  
  
    public void addEdge(int u, int v, int weight, boolean directed) {  
        graph.putIfAbsent(u, new ArrayList<>());  
        graph.get(u).add(new Edge(v, weight));  
  
        if (!directed) {  
            graph.putIfAbsent(v, new ArrayList<>());  
            graph.get(v).add(new Edge(u, weight));  
        }  
    }  
}
```

```

public void printGraph() {
    for (int u : graph.keySet()) {
        System.out.print(u + " -> ");
        for (Edge e : graph.get(u)) {
            System.out.print("(" + e.to + ", " + e.weight + ") ");
        }
        System.out.println();
    }
}

public static void main(String[] args) {
    WeightedGraph wg = new WeightedGraph();
    wg.addEdge(0, 1, 5, false);
    wg.addEdge(0, 2, 3, false);
    wg.addEdge(1, 3, 2, false);

    wg.printGraph();
}

```

Summary of Implementations

Type	Use Case	Space	Edge Lookup
Adjacency List	Most common, scalable	$O(V+E)$	$O(k)$
Adjacency Matrix	Dense graphs	$O(V^2)$	$O(1)$
Weighted Graph	Shortest paths, Dijkstra	$O(V+E)$	$O(k)$

✓ Why Use Weighted Graphs?

Use Case	Description
Dijkstra's Algorithm	Find shortest path based on cost
Prim's / Kruskal's Algorithm	Minimum Spanning Tree
A* Search	Weighted pathfinding with heuristics
Network Flow	Weights as capacities or bandwidth

d. Stack and Queue

- Clone Graph

You are given a reference to a node in a **connected undirected graph**. Return a **deep copy** (clone) of the graph.

Each node contains:

java

Copy

Edit

```
class Node {
    int val;
    List<Node> neighbors;
}
```

✓ Strategy to Solve:

Key Idea:

- Use a `HashMap<Node, Node>` to remember visited nodes and their clones.
- Perform DFS or BFS to visit each node and recursively clone its neighbors.

✓ DFS Solution in Java:

```
class Node {
    int val;
    List<Node> neighbors;

    Node(int val) {
        this.val = val;
        this.neighbors = new ArrayList<>();
    }
}
```

```
public class GraphCloner {
    private Map<Node, Node> visited = new HashMap<>();

    public Node cloneGraph(Node node) {
        if (node == null) return null;

        if (visited.containsKey(node))
            return visited.get(node);

        Node clone = new Node(node.val);
        visited.put(node, clone);

        for (Node neighbor : node.neighbors)
            clone.neighbors.add(cloneGraph(neighbor)); // recursive DFS

        return clone;
    }
}
```

✓ BFS Solution (Iterative):

```

public Node cloneGraphBFS(Node node) {
    if (node == null) return null;

    Map<Node, Node> visited = new HashMap<>();
    Queue<Node> queue = new LinkedList<>();

    Node clone = new Node(node.val);
    visited.put(node, clone);
    queue.offer(node);

    while (!queue.isEmpty()) {
        Node current = queue.poll();

```

```

        for (Node neighbor : current.neighbors) {
            if (!visited.containsKey(neighbor)) {
                visited.put(neighbor, new Node(neighbor.val));
                queue.offer(neighbor);
            }
            visited.get(current).neighbors.add(visited.get(neighbor));
        }
    }

    return clone;
}

```

e. Heap

- Sliding Window Maximum

✓ What is Sliding Window?

It's a technique to avoid unnecessary recalculations by maintaining a "window" of data and sliding it across the array or string.

✓ Template (Fixed-size Window)

java

Copy

Edit

```
public int maxSum(int[] nums, int k) {
    int maxSum = 0, windowSum = 0;

    for (int i = 0; i < nums.length; i++) {
        windowSum += nums[i];

        if (i >= k - 1) {
            maxSum = Math.max(maxSum, windowSum);
            windowSum -= nums[i - (k - 1)];
        }
    }

    return maxSum;
}
```



Example:

Input: nums = [2,1,5,1,3,2], k = 3

Output: 9 → subarray [5,1,3]

Problem: Longest Substring Without Repeating Characters

java

Copy

Edit

```
public int lengthOfLongestSubstring(String s) {
    Set<Character> seen = new HashSet<>();
    int left = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        while (seen.contains(s.charAt(right))) {
            seen.remove(s.charAt(left++)); // shrink from left
        }

        seen.add(s.charAt(right));
        maxLen = Math.max(maxLen, right - left + 1);
    }

    return maxLen;
}
```



✓ When to Use Sliding Window?

- Find max/min sum of **k** elements
- Longest substring with condition (e.g., no repeating chars)
- Count subarrays with certain sum
- All problems asking for “contiguous” or “subarray/substring” with optimal time

Great choice! "Kth Largest Element in an Array" is a classic problem that tests your understanding of **heaps**, **sorting**, and **priority queues** — often seen in interviews (LeetCode 215).

✓ Problem:

Given an integer array `nums` and an integer `k`, return the **kth largest element** in the array.

🔍 Example:

java

Copy

Edit

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: `5` // 2nd largest element



✓ Approach 1: Min-Heap (Best for large input)

🔧 Logic:

- Use a min-heap of size `k`.
- Keep inserting elements. If size exceeds `k`, remove the smallest.
- At the end, top of the heap is the **kth largest**.

java

Copy

Edit

```
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }

    return minHeap.peek();
}
```



✓ Approach 2: Sort and Pick

java

Copy

Edit

```
public int findKthLargestSort(int[] nums, int k) {
    Arrays.sort(nums);
    return nums[nums.length - k];
}
```

📌 **Note:** This is simple but slower ($O(n \log n)$), and not optimal for large inputs.

✓ Problem:

Given an array of integers and an integer `k`, return the `k` most frequent elements.

Example:

java

Copy

Edit

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

✓ Approach:

1. Use `HashMap` to count frequency of each number.
2. Use `Min Heap` to keep top `k` frequent elements.
3. Return elements from heap.

```
public class TopKFrequent {  
    public static int[] topKFrequent(int[] nums, int k) {  
        Map<Integer, Integer> freqMap = new HashMap<>();  
  
        // Step 1: Count frequency  
        for (int num : nums) {  
            freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);  
        }  
  
        // Step 2: Use min heap to keep top k elements  
        PriorityQueue<Map.Entry<Integer, Integer>> minHeap =  
            new PriorityQueue<>(Comparator.comparingInt(Map.Entry::getValue));  
  
        for (Map.Entry<Integer, Integer> entry : freqMap.entrySet()) {  
            minHeap.offer(entry);  
            if (minHeap.size() > k) minHeap.poll();  
        }  
  
        // Step 3: Extract keys from heap  
        int[] result = new int[k];  
        ↓  
    }  
}
```

```

// Step 3: Extract keys from heap
int[] result = new int[k];
int i = 0;
while (!minHeap.isEmpty()) {
    result[i++] = minHeap.poll().getKey();
}

return result;
}

public static void main(String[] args) {
    int[] nums = {1, 1, 1, 2, 2, 3};
    int k = 2;
    System.out.println(Arrays.toString(topKFrequent(nums, k))); // Output: [2, 1]
}
}

```

- Find duplicates in Array

✓ Problem:

Given an integer array `nums`, find and return all duplicate elements.

Example:

java

Copy

Edit

Input: `nums = [4, 3, 2, 7, 8, 2, 3, 1]`

Output: `[2, 3]`

✓ Approach 1: Using HashSet

🔧 Logic:

- Use a `Set` to track seen numbers.
- If a number is already in the set, it's a duplicate.

```
java Copy Edit  
  
public List<Integer> findDuplicates(int[] nums) {  
    Set<Integer> seen = new HashSet<>();  
    Set<Integer> duplicates = new HashSet<>();  
  
    for (int num : nums) {  
        if (!seen.add(num)) {  
            duplicates.add(num);  
        }  
    }  
  
    return new ArrayList<>(duplicates);  
}
```

✓ Approach 2: In-place Marking (Optimized for $1 \leq \text{nums}[i] \leq n$)

🧠 Trick:

- Use `nums[abs(num) - 1]` index and mark it negative.
- If it's already negative, it's a duplicate.

```
java Copy Edit  
  
public List<Integer> findDuplicatesOptimized(int[] nums) {  
    List<Integer> result = new ArrayList<>();  
  
    for (int i = 0; i < nums.length; i++) {  
        int index = Math.abs(nums[i]) - 1;  
  
        if (nums[index] < 0) {  
            result.add(Math.abs(nums[i]));  
        } else {  
            nums[index] = -nums[index];  
        }  
    }  
  
    return result;  
}
```